LAB REPORT

LAB 1: WEB CLASSIFIER

Môn học:	Phân tích dữ liệu truyền thông xã hội
Giảng viên hướng dẫn:	ThS. Lê Nhật Tùng
Họ và tên sinh viên:	Huỳnh Thái Linh
Mã số sinh viên:	2286400015

Mục lục

1 Giới thiệu **2**

2 Mục tiêu bài LAB **2**

3 Môi trường và công cụ thực hiện **3**

4 Mô tả mô hình sử dụng **4**

4.1 Thông tin chung về mô hình 4

4.2 Kiến trúc mô hình 4

5 Quy trình thực hiện **6**

5.1 Kết nối Google Drive 6

5.2 Cài đặt thư viện cần thiết 6

5.3 Import thư viện và load model 7

5.4 Phân tích thông tin đầu vào/đầu ra của mô hình 9

5.5 Xem kiến trúc mô hình 10

5.6 Xây dựng hàm tiền xử lý ảnh 11

5.7 Xây dựng Flask API 13

5.8 Xử lý logic dự đoán trong endpoint 14

5.9 Public server bằng ngrok 15

5.10 Chạy Flask server trên Colab 16

5.11 Xây dựng giao diện người dùng 16

6 Kết quả thực hiện **18**

6.1 Mô hình được nạp thành công 18

6.2 API hoạt động bình thường 18

6.3 Server được public thành công qua ngrok 18

6.4 Kết quả dự đoán ảnh 19

6.5 Giao diện kết quả dự đoán 19

7 Nhận xét **20**

7.1 Ưu điểm 20

7.2 Hạn chế 21

7.3 Bài học rút ra 21

8 Kết luận **22**

I. Giới thiệu

Trong lĩnh vực trí tuệ nhân tạo và học sâu (Deep Learning), bài toán phân loại ảnh là một trong những ứng dụng phổ biến và có ý nghĩa thực tiễn cao. Tuy nhiên, sau khi mô hình được huấn luyện xong, để có thể đưa vào sử dụng thực tế thì cần triển khai mô hình dưới dạng một dịch vụ cho phép người dùng hoặc các hệ thống khác gửi dữ liệu và nhận kết quả dự đoán. Một cách triển khai phổ biến hiện nay là xây dựng REST API.

Bài lab này trình bày quá trình triển khai một mô hình phân loại ảnh Cat/Dog đã được huấn luyện sẵn thành một API bằng Flask. Toàn bộ quá trình được thực hiện trên Google Colab và sử dụng ngrok để public server ra internet, cho phép truy cập API từ bên ngoài môi trường Colab.

Cụ thể, người dùng có thể gửi ảnh đến server thông qua API. Ảnh sau đó sẽ được tiền xử lý, đưa vào mô hình để dự đoán, và kết quả trả về dưới dạng JSON gồm nhãn phân loại (Cat hoặc Dog) cùng với độ tin cậy tương ứng. Bài thực hành giúp làm rõ quy trình triển khai mô hình học sâu thành một ứng dụng có khả năng phục vụ dự đoán trong thực tế.

II. Mục tiêu bài LAB

Bài lab này được thực hiện nhằm đạt được các mục tiêu sau:

- Kết nối Google Drive với Google Colab để truy cập và sử dụng mô hình đã được huấn luyện trước.
- Cài đặt và sử dụng các thư viện cần thiết cho quá trình triển khai hệ thống, bao gồm Flask, Flask-CORS, pyngrok, Pillow và TensorFlow.
- Nạp mô hình học sâu từ file định dạng '.h5' và xác định các thông tin quan trọng của mô hình như kích thước đầu vào, kích thước đầu ra và kiểu phân loại.
- Xây dựng một REST API bằng Flask với các endpoint phục vụ việc kiểm tra trạng thái hệ thống và thực hiện dự đoán ảnh.
- Thiết kế hàm tiền xử lý ảnh nhằm chuyển đổi dữ liệu đầu vào về đúng định dạng mà mô hình yêu cầu trước khi đưa vào dự đoán.
- Triển khai và công khai API ra internet thông qua ngrok, giúp hệ thống có thể được truy cập từ bên ngoài môi trường Google Colab.

- Kiểm tra hoạt động của API và đánh giá khả năng dự đoán ảnh Cat/Dog thông qua các công cụ như Postman, trình duyệt hoặc lệnh `curl`.

III. Môi trường và công cụ thực hiện

Bài lab được triển khai trên nền tảng Google Colab với ngôn ngữ lập trình Python và các thư viện hỗ trợ cho việc nạp mô hình, xử lý ảnh, xây dựng API và đưa hệ thống ra môi trường internet. Các công cụ và thư viện chính được sử dụng trong bài lab được trình bày trong Bảng 1.

Bảng 1: Danh sách công cụ và thư viện sử dụng trong bài lab

Công cụ / Thư viện	**Mô tả**
Google Colab	Môi trường chạy notebook trên nền tảng đám mây, hỗ trợ thực thi mã Python thuận tiện.
Python 3.12	Ngôn ngữ lập trình chính được sử dụng trong bài lab.
TensorFlow / Keras	Framework học sâu dùng để nạp và thực thi mô hình đã huấn luyện.
Flask	Framework web gọn nhẹ của Python, được sử dụng để xây dựng REST API.
Flask-CORS	Thư viện hỗ trợ xử lý các yêu cầu truy cập khác miền (CORS).
pyngrok	Thư viện Python dùng để tạo kết nối tunnel thông qua ngrok.
Pillow (PIL)	Thư viện hỗ trợ đọc và xử lý ảnh.
NumPy	Thư viện hỗ trợ tính toán số học và xử lý mảng dữ liệu.
Google Drive	Nơi lưu trữ file mô hình `.h5` đã được huấn luyện trước.
ngrok	Dịch vụ tạo đường hầm mạng giúp public server cục bộ ra internet.

Lưu ý rằng Google Colab đã tích hợp sẵn một số thư viện cơ bản như TensorFlow, NumPy và Pillow. Trong quá trình thực hiện, chỉ cần cài đặt bổ sung một số thư viện phục vụ cho việc xây dựng và triển khai API như Flask-CORS và pyngrok.

IV. Mô tả mô hình sử dụng

4.1 Thông tin chung về mô hình

Mô hình được sử dụng trong bài lab là một mạng nơ-ron tích chập (Convolutional Neural Network - CNN) đã được huấn luyện trước cho bài toán phân loại ảnh Cat/Dog. Mô hình được lưu dưới định dạng `.h5` của Keras/TensorFlow và được nạp trực tiếp từ Google Drive trong môi trường Google Colab.

Sau khi nạp mô hình, một số thông tin quan trọng được xác định như sau:

- Định dạng lưu trữ mô hình: `.h5`
- Kích thước đầu vào (Input Shape): `(None, 128, 128, 3)`
- Kích thước đầu ra (Output Shape): `(None, 1)`
- Loại bài toán: Phân loại nhị phân (Binary Classification)
- Nhãn lớp: `Cat` và `Dog`

Trong đó, kích thước đầu vào `(None, 128, 128, 3)` cho thấy mô hình tiếp nhận ảnh màu RGB có kích thước 128×128 pixel. Thành phần `None` biểu thị số lượng mẫu trong mỗi batch là linh hoạt và không cố định trước.

Đầu ra của mô hình có dạng `(None, 1)`, tương ứng với một giá trị xác suất duy nhất được tạo ra bởi hàm kích hoạt Sigmoid. Vì đây là bài toán phân loại nhị phân, giá trị đầu ra được sử dụng để xác định nhãn dự đoán theo ngưỡng 0.5. Cụ thể, nếu xác suất đầu ra nhỏ hơn 0.5 thì ảnh được gán nhãn `Cat`; ngược lại, nếu giá trị lớn hơn hoặc bằng 0.5, ảnh sẽ được gán nhãn `Dog`.

4.2 Kiến trúc mô hình

Sau khi nạp thành công, mô hình được hiển thị với tên `sequential_3`. Tổng số tham số của mô hình là 19,452,163, tương đương khoảng 74.20 MB. Trong đó, số tham số có thể huấn luyện là 19,450,241 và số tham số không huấn luyện là 1,920.

Kiến trúc của mô hình được xây dựng theo dạng mạng CNN nhiều tầng, bao gồm các lớp tích chập, chuẩn hóa, gộp cực đại, dropout và các lớp kết nối đầy đủ. Cấu trúc này cho phép mô hình trích xuất đặc trưng từ ảnh đầu vào theo từng mức độ từ thấp đến cao, đồng thời giảm hiện tượng overfitting trong quá trình huấn luyện.

Cụ thể, mô hình gồm ba khối tích chập chính. Mỗi khối bao gồm các lớp Conv2D để trích xuất đặc trưng ảnh, tiếp theo là lớp BatchNormalization nhằm ổn định dữ liệu trung gian, lớp MaxPooling2D để giảm kích thước không gian của đặc trưng, và lớp

Dropout để hạn chế overfitting. Sau ba khối tích chập, đầu ra được làm phẳng bằng lớp Flatten trước khi đưa vào lớp Dense 512 neuron. Cuối cùng, mô hình sử dụng một lớp Dense với 1 neuron và hàm kích hoạt Sigmoid để sinh ra xác suất dự đoán cho bài toán phân loại nhị phân.

Bảng 2 trình bày chi tiết các lớp trong mô hình cùng với kích thước đầu ra và số lượng tham số tương ứng.

Bảng 2: Chi tiết kiến trúc mô hình CNN sử dụng trong bài lab

Layer (Lớp)	**Output Shape**	**Số tham số**
Conv2D-0 (Conv2D)	(None, 126, 126, 32)	896
Conv2D-1 (Conv2D)	(None, 124, 124, 64)	18,496
BatchNormalization-2	(None, 124, 124, 64)	256
MaxPooling2D-3	(None, 62, 62, 64)	0
Dropout-4	(None, 62, 62, 64)	0
Conv2D-5 (Conv2D)	(None, 60, 60, 64)	36,928
Conv2D-6 (Conv2D)	(None, 58, 58, 128)	73,856
BatchNormalization-7	(None, 58, 58, 128)	512
MaxPooling2D-8	(None, 29, 29, 128)	0
Dropout-9	(None, 29, 29, 128)	0
Conv2D-10 (Conv2D)	(None, 27, 27, 128)	147,584
Conv2D-11 (Conv2D)	(None, 25, 25, 256)	295,168
BatchNormalization-12	(None, 25, 25, 256)	1,024
MaxPooling2D-13	(None, 12, 12, 256)	0
Dropout-14	(None, 12, 12, 256)	0
Flatten-15	(None, 36864)	0
Dense-16 (Dense)	(None, 512)	18,874,880
BatchNormalization-17	(None, 512)	2,048
Dropout-18	(None, 512)	0
Dense-19 (Dense)	(None, 1)	513

Tổng số tham số: 19,452,163 (74.20 MB)

Số tham số có thể huấn luyện: 19,450,241 (74.20 MB)

Số tham số không huấn luyện: 1,920 (7.50 KB)

Nhìn chung, mô hình có cấu trúc tương đối sâu và phù hợp với bài toán phân loại ảnh nhị phân. Các lớp tích chập giúp mô hình học được đặc trưng quan trọng của ảnh

mèo và chó, trong khi các lớp BatchNormalization và Dropout góp phần cải thiện tính ổn định và khả năng tổng quát hóa của mô hình. Lớp đầu ra sử dụng hàm Sigmoid là lựa chọn phù hợp cho bài toán chỉ gồm hai lớp.

V. Quy trình thực hiện

5.1 Kết nối Google Drive

Bước đầu tiên của bài lab là kết nối Google Drive với môi trường Google Colab nhằm truy cập file mô hình đã được huấn luyện và lưu trữ trước đó.

Code thực hiện:

```
from google.colab import drive
drive.mount('/content/drive')
```

Giải thích: Lệnh `drive.mount('/content/drive')` cho phép liên kết Google Drive với thư mục làm việc của Colab. Sau khi người dùng cấp quyền truy cập, toàn bộ dữ liệu trên Google Drive sẽ được gắn vào đường dẫn `/content/drive`. Việc kết nối này giúp hệ thống có thể truy cập đến file mô hình `model.h5` đã được lưu trữ từ trước.

Trong bài lab này, mô hình được lưu tại đường dẫn: `/content/drive/MyDrive/Colab Notebooks/model/model.h5`

Kết quả: Sau khi thực hiện lệnh trên, Colab hiển thị thông báo như `Mounted at /content/drive` hoặc `Drive already mounted at /content/drive`, cho thấy quá trình kết nối đã thành công.

Hình 1: Kết nối Google Drive trên Google Colab

5.2 Cài đặt thư viện cần thiết

Sau khi kết nối Google Drive, bước tiếp theo là cài đặt các thư viện cần thiết cho quá trình triển khai API. Mặc dù Google Colab đã tích hợp sẵn một số thư viện cơ bản

như TensorFlow, Pillow và NumPy, việc khai báo đầy đủ trong lệnh cài đặt giúp đảm bảo môi trường chạy được chuẩn bị đầy đủ và đồng nhất.

Code thực hiện:

```
!pip install flask flask-cors pyngrok pillow tensorflow
```

Giải thích các thư viện sử dụng:

- `flask`: framework web gọn nhẹ, dùng để xây dựng REST API.
- `flask-cors`: thư viện hỗ trợ xử lý CORS, cho phép các ứng dụng ở domain khác có thể truy cập API.
- `pyngrok`: thư viện Python hỗ trợ tạo tunnel ngrok để public server.
- `pillow`: thư viện dùng để đọc, mở, resize và chuyển đổi ảnh.
- `tensorflow`: framework học sâu được sử dụng để nạp và thực thi mô hình.

Kết quả: Sau khi cài đặt thành công, Colab hiển thị thông báo như `Successfully installed flask-cors ... pyngrok ...`, xác nhận các thư viện bổ sung đã được cài đặt đầy đủ.

```
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/l
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/py
Downloading flask_cors-6.0.2-py3-none-any.whl (13 kB)
Downloading pyngrok-7.5.1-py3-none-any.whl (24 kB)
Installing collected packages: pyngrok, flask-cors
Successfully installed flask-cors-6.0.2 pyngrok-7.5.1
```

Hình 2: Cài đặt các thư viện cần thiết cho hệ thống

5.3 Import thư viện và load model

Sau khi chuẩn bị xong môi trường làm việc, hệ thống tiến hành import các thư viện cần thiết và nạp mô hình phân loại ảnh đã được huấn luyện trước.

Bước 1 – Import thư viện:

```
from flask import Flask, request, jsonify
from flask_cors import CORS
from PIL import Image
import base64
import numpy as np
import io
import threading
```

Các thư viện trên phục vụ cho việc xây dựng API, xử lý ảnh, thao tác dữ liệu dạng mảng, giải mã dữ liệu ảnh base64 và chạy Flask server trong nền.

Bước 2 – Nạp mô hình:

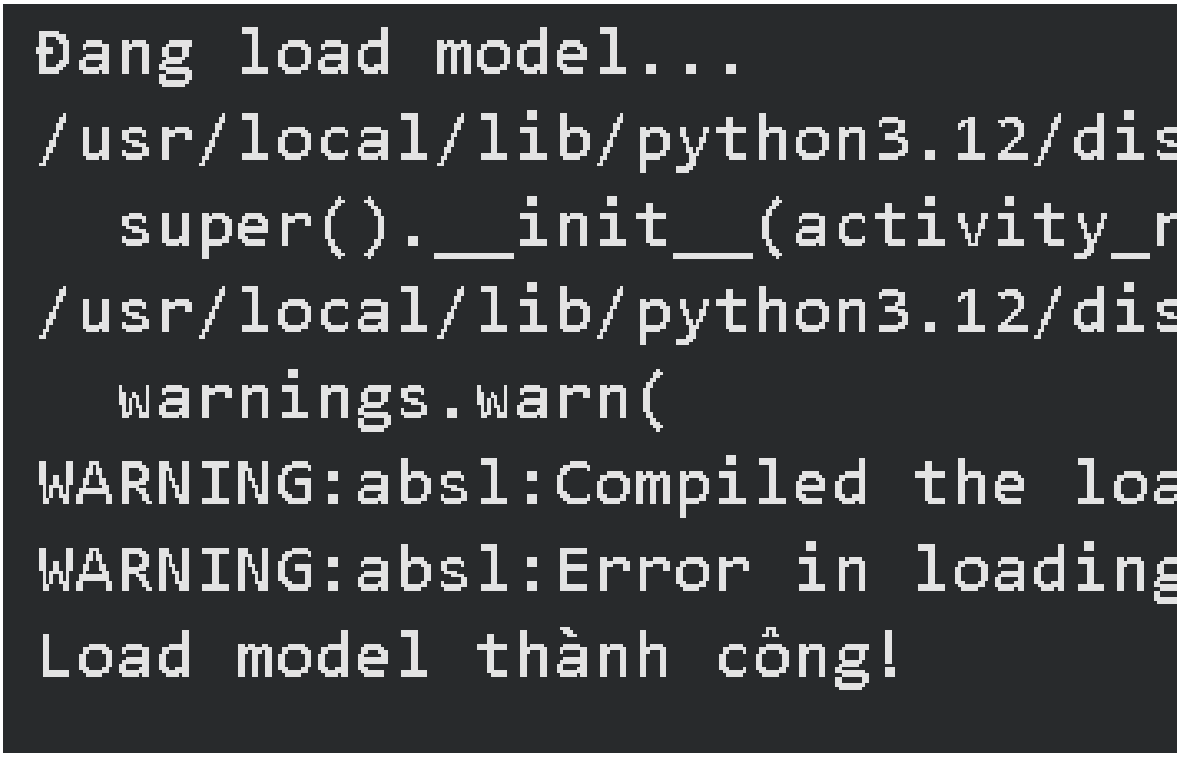
```
import tensorflow as tf
from tensorflow.keras.models import load_model

MODEL_PATH = '/content/drive/MyDrive/Colab Notebooks/model/
    model.h5'

print("Dang load model...")
model = load_model(MODEL_PATH)
print("Load model thanh cong!")
```

Giải thích: Biến `MODEL_PATH` chứa đường dẫn đến file mô hình định dạng `.h5`. Hàm `load_model()` của Keras được sử dụng để nạp toàn bộ mô hình, bao gồm kiến trúc mạng và các trọng số đã được huấn luyện. Sau khi quá trình này hoàn tất, mô hình sẵn sàng cho bước dự đoán.

Kết quả: Màn hình hiển thị lần lượt thông báo `Đang load model...` và `Load model thành công!`. Trong một số trường hợp có thể xuất hiện thêm cảnh báo liên quan đến phiên bản Keras hoặc optimizer, tuy nhiên các cảnh báo này không ảnh hưởng đến khả năng dự đoán của mô hình.



Hình 3: Quá trình nạp mô hình từ file .h5

5.4 Phân tích thông tin đầu vào/đầu ra của mô hình

Sau khi nạp mô hình thành công, bước tiếp theo là xác định kích thước đầu vào và đầu ra của mô hình. Đây là cơ sở quan trọng để xây dựng hàm tiền xử lý ảnh đúng với định dạng mà mô hình yêu cầu.

Code thực hiện:

```
input_shape = model.input_shape
print(f"Input Shape: {input_shape}")

if len(input_shape) == 4:
    IMG_HEIGHT = input_shape[1]
    IMG_WIDTH = input_shape[2]
    CHANNELS = input_shape[3]
else:
    IMG_HEIGHT = input_shape[1]
    IMG_WIDTH = input_shape[2]
    CHANNELS = 3

print(f"Kich thuoc anh: {IMG_WIDTH} x {IMG_HEIGHT}")
print(f"So channels: {CHANNELS}")

output_shape = model.output_shape
print(f"Output Shape: {output_shape}")

if len(output_shape) == 2:
    num_classes = output_shape[1]
else:
    num_classes = output_shape[-1]

print(f"So classes: {num_classes}")

if num_classes == 1:
    print("Model type: Binary Classification (Sigmoid)")
    CLASS_NAMES = ['Cat', 'Dog']
else:
    print("Model type: Multi-class Classification (Softmax)")
    CLASS_NAMES = ['Cat', 'Dog']

print(f"Class names: {CLASS_NAMES}")
```

Giải thích: Giá trị `model.input_shape` cho biết mô hình nhận đầu vào có kích thước `(None, 128, 128, 3)`, nghĩa là ảnh màu RGB có kích thước 128×128 pixel. Từ đó, các biến `IMG_HEIGHT`, `IMG_WIDTH` và `CHANNELS` được gán tự động để sử dụng trong bước tiền xử lý ảnh.

Tương tự, `model.output_shape` cho biết đầu ra của mô hình có dạng `(None, 1)`. Điều này cho thấy mô hình đang xử lý bài toán phân loại nhị phân và sử dụng một giá trị xác suất duy nhất để đưa ra quyết định phân loại. Danh sách nhãn lớp được gán tương ứng là `Cat` và `Dog`.

Kết quả hiển thị:

- Input Shape: `(None, 128, 128, 3)`
- Kích thước ảnh: `128 x 128`
- Số channels: `3`
- Output Shape: `(None, 1)`
- Số classes: `1`
- Model type: `Binary Classification (Sigmoid)`
- Class names: `['Cat', 'Dog']`

```
Input Shape: (None, 128, 128, 3)
Kích thước ảnh: 128 x 128
Số channels: 3

Output Shape: (None, 1)
Số classes: 1
Model type: Binary Classification (Sigmoid)
Class names: ['Cat', 'Dog']
```

Hình 4: Thông tin đầu vào và đầu ra của mô hình phân loại

5.5 Xem kiến trúc mô hình

Để quan sát tổng quan cấu trúc của mô hình, hàm `model.summary()` được sử dụng nhằm hiển thị danh sách các lớp, kích thước đầu ra và số lượng tham số của từng lớp.

Code thực hiện:

```
print("MODEL SUMMARY:")
model.summary()
```

Giải thích: Kết quả từ `model.summary()` cho thấy mô hình được xây dựng theo kiến trúc CNN nhiều tầng với các lớp tích chập, chuẩn hóa, gộp cực đại, dropout và các lớp kết nối đầy đủ. Nội dung chi tiết của kiến trúc mô hình đã được trình bày tại Mục 4.2.

Từ bảng tóm tắt mô hình có thể thấy rằng:

- mô hình gồm nhiều lớp tích chập để trích xuất đặc trưng ảnh;
- các lớp BatchNormalization được sử dụng để ổn định quá trình xử lý dữ liệu;
- các lớp Dropout được thêm vào để giảm hiện tượng overfitting;
- lớp Dense với 512 neuron chiếm phần lớn số lượng tham số của mô hình;
- lớp đầu ra chỉ gồm 1 neuron với hàm kích hoạt Sigmoid, phù hợp với bài toán phân loại nhị phân.

5.6 Xây dựng hàm tiền xử lý ảnh

Trước khi ảnh được đưa vào mô hình để dự đoán, cần thực hiện bước tiền xử lý nhằm đảm bảo dữ liệu đầu vào có đúng kích thước, đúng số kênh màu và đúng định dạng mà mô hình yêu cầu.

Code thực hiện:

```
def preprocess_image(image_bytes):
    image = Image.open(io.BytesIO(image_bytes))

    if CHANNELS == 3:
        image = image.convert('RGB')
    elif CHANNELS == 1:
        image = image.convert('L')

    image = image.resize((IMG_WIDTH, IMG_HEIGHT))
    img_array = np.array(image)
    img_array = img_array / 255.0

    if CHANNELS == 1:
        img_array = np.expand_dims(img_array, axis=-1)
```

```
    img_array = np.expand_dims(img_array, axis=0)
    return img_array
```

Giải thích: Hàm `preprocess_image()` nhận vào dữ liệu ảnh ở dạng bytes, sau đó thực hiện lần lượt các bước: mở ảnh bằng Pillow, chuyển đổi không gian màu phù hợp với số kênh mà mô hình yêu cầu, resize về kích thước `128x128`, chuyển sang mảng NumPy, chuẩn hóa giá trị pixel về khoảng $[0, 1]$, và cuối cùng thêm chiều batch để tạo đầu vào đúng định dạng cho mô hình.

Đối với mô hình hiện tại, ảnh đầu vào sau xử lý sẽ có kích thước `(1, 128, 128, 3)`.

```python
def preprocess_image(image_bytes):
    # Mở ảnh
    image = Image.open(io.BytesIO(image_bytes))

    # Convert sang RGB hoặc Grayscale
    if CHANNELS == 3:
        image = image.convert('RGB')
    elif CHANNELS == 1:
        image = image.convert('L')

    # Resize theo kích thước model yêu cầu
    image = image.resize((IMG_WIDTH, IMG_HEIGHT))

    # Convert sang array
    img_array = np.array(image)

    # Normalize về 0-1
    img_array = img_array / 255.0

    # Nếu grayscale, thêm channel dimension
    if CHANNELS == 1:
        img_array = np.expand_dims(img_array, axis=-1)

    # Thêm batch dimension
    img_array = np.expand_dims(img_array, axis=0)

    return img_array
```

Hình 5: Hàm tiền xử lý ảnh trước khi đưa vào mô hình

5.7 Xây dựng Flask API

Sau khi mô hình và bước tiền xử lý đã được chuẩn bị, hệ thống tiến hành xây dựng REST API bằng Flask để cung cấp dịch vụ dự đoán ảnh qua giao thức HTTP. API bao gồm các endpoint chính dùng để kiểm tra trạng thái hệ thống và thực hiện dự đoán.

Khởi tạo ứng dụng Flask:

```
app = Flask(__name__)
CORS(app)
```

Lệnh `Flask(__name__)` dùng để khởi tạo ứng dụng Flask, trong khi `CORS(app)` cho phép các ứng dụng ở domain khác có thể truy cập API mà không bị chặn bởi cơ chế bảo mật của trình duyệt.

Endpoint /:

```
@app.route('/')
def home():
    return jsonify({
        'status': 'running',
        'message': 'Dog Cat Classifier API',
        'model_info': {
            'input_size': f'{IMG_WIDTH}x{IMG_HEIGHT}',
            'channels': CHANNELS,
            'classes': CLASS_NAMES
        }
    })
```

Endpoint gốc `/` trả về thông tin tổng quan của hệ thống, bao gồm trạng thái server, tên API và cấu hình đầu vào của mô hình.

Endpoint `/health`:

```
@app.route('/health', methods=['GET'])
def health():
    return jsonify({
        'status': 'healthy',
        'model_loaded': True,
        'input_size': f'{IMG_WIDTH}x{IMG_HEIGHT}'
    })
```

Endpoint `/health` được sử dụng để kiểm tra tình trạng hoạt động của server và xác nhận mô hình đã được nạp thành công.

5.8 Xử lý logic dự đoán trong endpoint

Endpoint `/predict` là thành phần chính của hệ thống, đảm nhiệm việc tiếp nhận ảnh từ phía client, tiền xử lý dữ liệu, thực hiện dự đoán bằng mô hình và trả về kết quả dưới dạng JSON.

Code thực hiện:

```
@app.route('/predict', methods=['POST'])
def predict():
    try:
        if 'file' in request.files:
            file = request.files['file']
            image_bytes = file.read()
        elif request.json and 'image' in request.json:
            image_data = request.json['image']
            if ',' in image_data:
                image_data = image_data.split(',')[1]
            image_bytes = base64.b64decode(image_data)
        else:
            return jsonify({'error': 'No image provided'}), 400

        processed_image = preprocess_image(image_bytes)
        prediction = model.predict(processed_image, verbose=0)

        if num_classes == 1:
            prob = float(prediction[0][0])
            if prob > 0.5:
                predicted_class = CLASS_NAMES[1]
                confidence = prob
            else:
                predicted_class = CLASS_NAMES[0]
                confidence = 1 - prob
        else:
            predicted_idx = np.argmax(prediction[0])
            predicted_class = CLASS_NAMES[predicted_idx]
            confidence = float(prediction[0][predicted_idx])

        return jsonify({
            'success': True,
            'prediction': predicted_class,
            'confidence': round(confidence * 100, 2),
```

```
            'raw_output': prediction[0].tolist()
        })

    except Exception as e:
        return jsonify({
            'success': False,
            'error': str(e)
        }), 500
```

Giải thích: Endpoint này hỗ trợ hai cách gửi dữ liệu ảnh: tải file trực tiếp thông qua `multipart/form-data` hoặc gửi chuỗi ảnh đã mã hóa base64 trong nội dung JSON. Sau khi nhận ảnh, hệ thống gọi hàm `preprocess_image()` để chuẩn hóa dữ liệu đầu vào trước khi đưa vào mô hình.

Đối với bài toán hiện tại, mô hình là phân loại nhị phân nên đầu ra chỉ gồm một xác suất duy nhất. Nếu giá trị xác suất lớn hơn 0.5, ảnh được gán nhãn `Dog`; ngược lại, ảnh được gán nhãn `Cat`. Kết quả cuối cùng được trả về dưới dạng JSON, bao gồm trạng thái xử lý, nhãn dự đoán, độ tin cậy và giá trị đầu ra thô của mô hình.

Nếu trong quá trình xử lý xảy ra lỗi, API sẽ trả về thông báo lỗi kèm mã trạng thái HTTP phù hợp.

5.9 Public server bằng ngrok

Do Google Colab hoạt động trong môi trường máy ảo và không cung cấp trực tiếp địa chỉ IP public, cần sử dụng ngrok để tạo một đường hầm mạng từ internet tới server cục bộ đang chạy trên Colab. Về cách tạo Authtoken cho ngrok thì truy cập vào web: https://dashboard.ngrok.com/authtokens nhấn "Add Authtoken" là sẽ được.

Code thực hiện:

```
NGROK_AUTH_TOKEN = 'YOUR_NGROK_AUTHTOKEN'
from pyngrok import ngrok

ngrok.set_auth_token(NGROK_AUTH_TOKEN)

port = 5000
public_url = ngrok.connect(port)
```

Giải thích: Trước tiên, hệ thống thiết lập `auth token` để xác thực với dịch vụ ngrok. Sau đó, lệnh `ngrok.connect(port)` tạo một tunnel từ một URL public đến cổng `5000` của server cục bộ. URL này cho phép người dùng hoặc các ứng dụng bên ngoài có thể truy cập API từ internet.

Trong phiên bản miễn phí, địa chỉ URL do ngrok cung cấp thường được tạo ngẫu nhiên và có thể thay đổi sau mỗi lần chạy lại notebook.

```
SERVER ĐANG CHẠY!
Public URL: NgrokTunnel: "https://fidel-euchromatic-sheryl.ngrok-free.dev" -> "http://localhost:5000"
```

Hình 6: Public Flask server ra internet bằng ngrok

5.10 Chạy Flask server trên Colab

Bước cuối cùng là khởi động Flask server để hệ thống bắt đầu tiếp nhận request từ bên ngoài. Do môi trường Google Colab hoạt động theo cơ chế notebook, Flask server được chạy trong một luồng riêng nhằm tránh làm gián đoạn quá trình thực thi các cell khác.

Code thực hiện:

```python
print("SERVER DANG CHAY!")
print(f"Public URL: {public_url}")

def run():
    app.run(port=port)

threading.Thread(target=run).start()
```

Giải thích: Hàm `run()` được xây dựng để khởi động Flask server tại cổng `5000`. Sau đó, `threading.Thread(target=run).start()` được sử dụng để chạy server trong nền. Cách làm này giúp server hoạt động liên tục trong khi notebook vẫn có thể tiếp tục thực hiện các thao tác khác.

Kết quả: Sau khi thực thi, màn hình hiển thị thông báo `SERVER ĐANG CHẠY!` cùng với `Public URL` do ngrok cung cấp. Đây chính là địa chỉ có thể được sử dụng để truy cập API từ bên ngoài.

5.11 Xây dựng giao diện người dùng

Để hỗ trợ việc tương tác với hệ thống một cách trực quan hơn, bài lab xây dựng thêm một giao diện người dùng đơn giản bằng HTML, CSS và JavaScript. Giao diện này đóng vai trò là frontend, cho phép người dùng nhập địa chỉ API, tải ảnh từ máy tính lên, gửi yêu cầu dự đoán và xem kết quả trực tiếp trên trình duyệt.

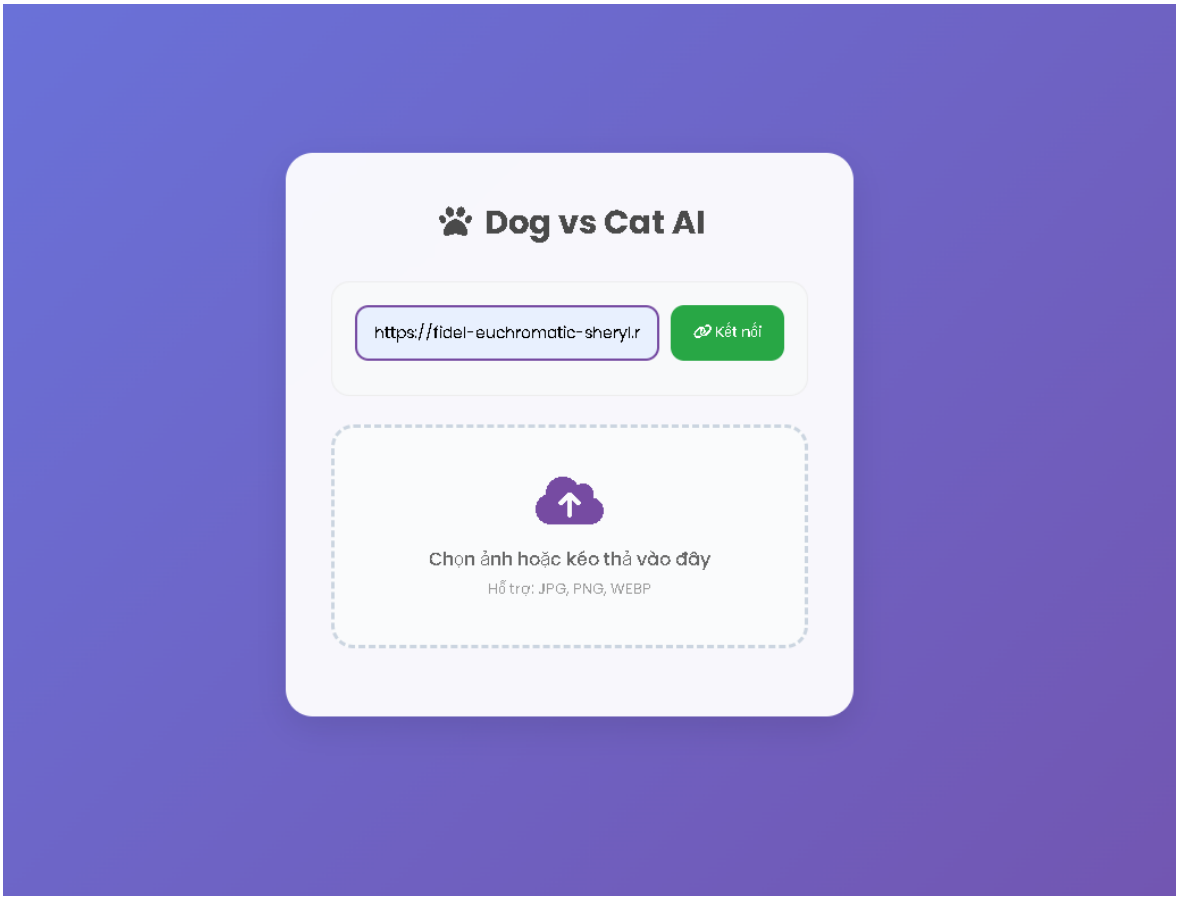
Giao diện được thiết kế theo hướng hiện đại, đơn giản và dễ sử dụng. Các thành phần chính của giao diện bao gồm:

- Ô nhập địa chỉ API: cho phép người dùng dán URL public do ngrok cung cấp.

- Nút kiểm tra kết nối: gửi yêu cầu đến endpoint `/health` để xác nhận API đang hoạt động.
- Vùng tải ảnh: hỗ trợ chọn ảnh từ máy tính hoặc kéo-thả ảnh vào giao diện.
- Khung xem trước ảnh: hiển thị ảnh đã chọn trước khi gửi đi dự đoán.
- Nút dự đoán: gửi ảnh đến endpoint `/predict` bằng phương thức `POST`.
- Khu vực hiển thị kết quả: hiển thị nhãn dự đoán (`Cat` hoặc `Dog`) cùng với độ tin cậy tương ứng.
- Nút làm mới: cho phép xóa kết quả hiện tại và thử lại với ảnh khác.

Phần CSS được sử dụng để tạo giao diện trực quan với các thành phần như hộp nhập liệu, nút bấm, vùng kéo-thả ảnh, hiệu ứng loading và khu vực hiển thị kết quả. JavaScript đảm nhiệm vai trò xử lý sự kiện người dùng, đọc file ảnh, kiểm tra kết nối API, gửi request đến server và cập nhật kết quả trả về lên giao diện.

Nhờ có frontend này, hệ thống không chỉ dừng lại ở mức API backend mà đã có thể hoạt động như một ứng dụng web đơn giản, giúp việc kiểm thử và trình diễn mô hình trở nên thuận tiện hơn.



Hình 7: Giao diện web của hệ thống phân loại ảnh Cat/Dog

VI. Kết quả thực hiện

Sau khi hoàn thành các bước triển khai, hệ thống đã hoạt động ổn định và đạt được các kết quả chính như sau.

6.1 Mô hình được nạp thành công

Mô hình CNN phục vụ cho bài toán phân loại ảnh Cat/Dog đã được nạp thành công từ file `.h5` lưu trên Google Drive. Kết quả kiểm tra cho thấy mô hình tiếp nhận ảnh đầu vào có kích thước `128 x 128 x 3` và đầu ra là một giá trị xác suất duy nhất thông qua hàm kích hoạt Sigmoid, phù hợp với bài toán phân loại nhị phân.

Ngoài ra, thông tin từ `model.summary()` cho thấy mô hình có tổng cộng 19,452,163 tham số, trong đó phần lớn tham số tập trung ở các lớp Dense phía cuối mạng.

6.2 API hoạt động bình thường

Sau khi xây dựng và khởi chạy Flask server, hệ thống đã vận hành đúng chức năng trên cổng `5000`. Các endpoint chính đều hoạt động ổn định:

- Endpoint `/` trả về thông tin tổng quan của API và mô hình;
- Endpoint `/health` trả về trạng thái `healthy`, xác nhận server đang hoạt động bình thường;
- Endpoint `/predict` tiếp nhận ảnh đầu vào và trả về kết quả dự đoán dưới dạng JSON.

Kết quả này cho thấy mô hình đã được tích hợp thành công vào một dịch vụ web có thể tiếp nhận và xử lý yêu cầu từ bên ngoài.

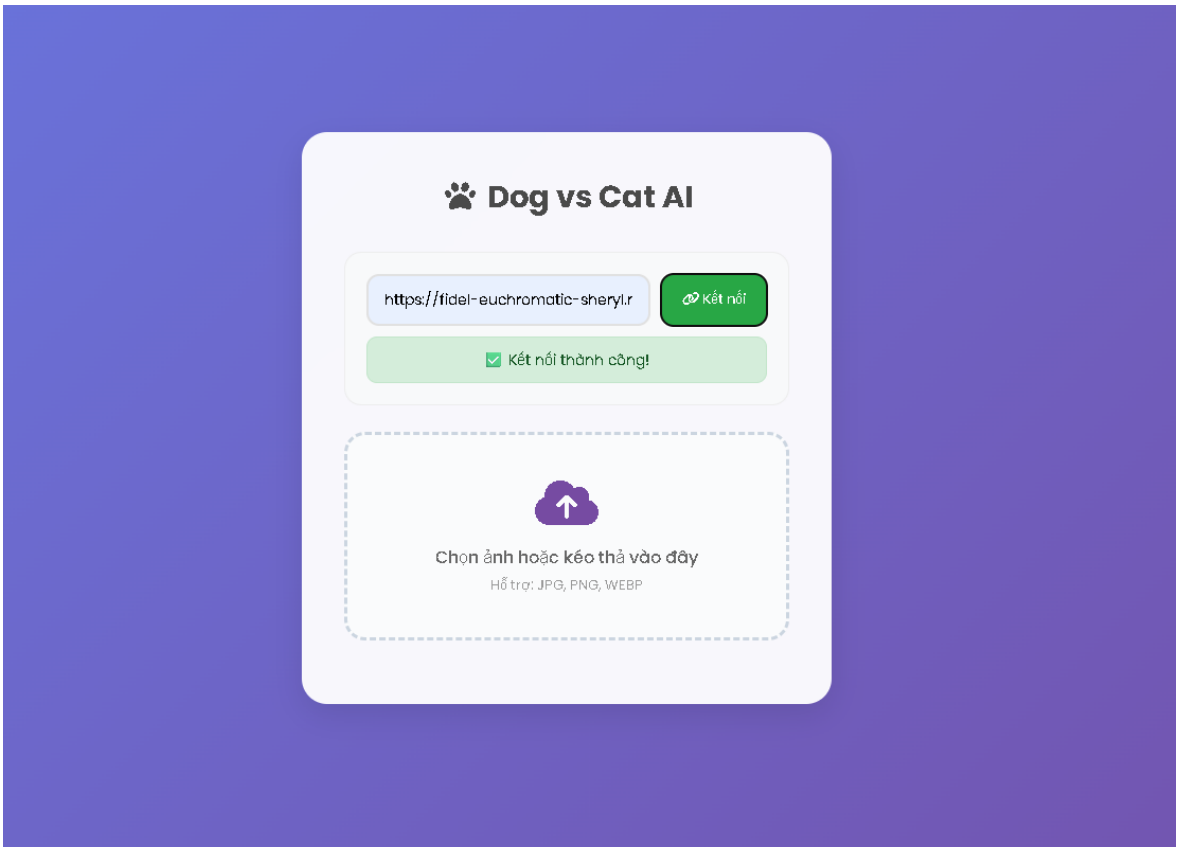
6.3 Server được public thành công qua ngrok

Thông qua thư viện `pyngrok`, một tunnel ngrok đã được tạo thành công để ánh xạ từ địa chỉ public trên internet đến Flask server đang chạy cục bộ trong Google Colab. Nhờ đó, API có thể được truy cập từ các thiết bị bên ngoài mà không cần triển khai trên một máy chủ vật lý riêng.

Trong lần chạy thử nghiệm, hệ thống nhận được một địa chỉ public dạng:

`https://fidel-euchromatic-sheryl.ngrok-free.dev`

Cần lưu ý rằng với phiên bản miễn phí của ngrok, địa chỉ này có thể thay đổi sau mỗi lần khởi động lại notebook.



Hình 8: Kết nối public server bằng ngrok

6.4 Kết quả dự đoán ảnh

Kết quả kiểm thử cho thấy API thực hiện đúng quy trình xử lý ảnh đầu vào, bao gồm tiếp nhận ảnh, tiền xử lý, đưa vào mô hình dự đoán và trả về phản hồi dưới dạng JSON. Thông tin phản hồi bao gồm nhãn dự đoán (`Cat` hoặc `Dog`) và độ tin cậy (`confidence`) tính theo phần trăm.

Ví dụ, khi gửi ảnh đầu vào là ảnh một con chó, API có thể trả về kết quả:

```
{"prediction": "Dog", "confidence": 98.52}
```

Tương tự, khi gửi ảnh đầu vào là ảnh một con mèo, API có thể trả về:

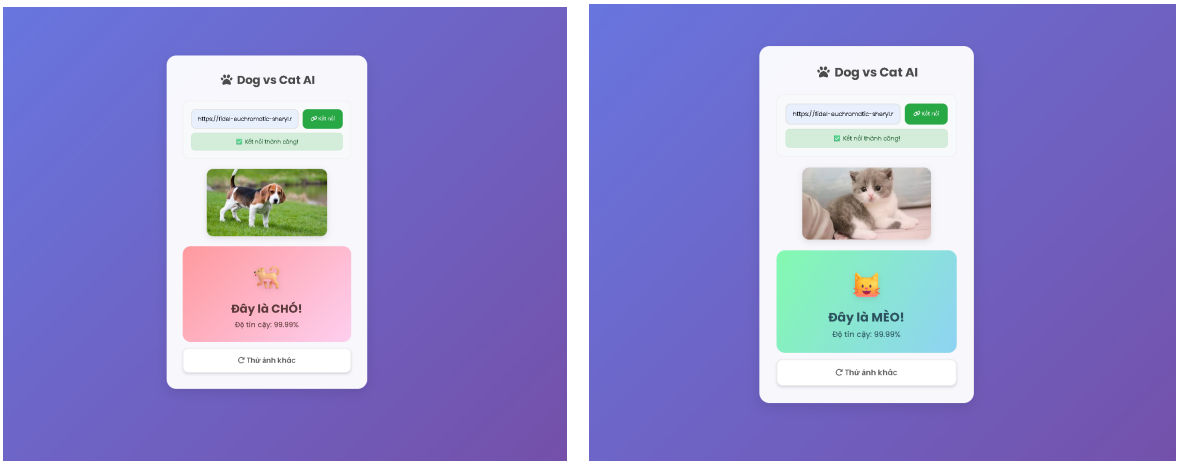
```
{"prediction": "Cat", "confidence": 95.30}
```

Các kết quả trên cho thấy hệ thống đã hoạt động đúng mục tiêu đề ra và có khả năng phục vụ cho các ứng dụng dự đoán ảnh thông qua giao tiếp API.

6.5 Giao diện kết quả dự đoán

Ngoài việc kiểm thử trực tiếp thông qua API, hệ thống còn được tích hợp với một giao diện web đơn giản viết bằng HTML, CSS và JavaScript. Giao diện này cho phép người

dùng nhập địa chỉ API, tải ảnh từ máy tính và xem kết quả dự đoán trực tiếp trên trình duyệt. Kết quả hiển thị bao gồm nhãn phân loại và độ tin cậy, giúp quá trình kiểm thử và trình diễn hệ thống trở nên trực quan hơn.



Hình 9: Hiển thị kết quả dự đoán

VII. Nhận xét

Sau quá trình thực hiện bài lab, có thể rút ra một số nhận xét và đánh giá về hệ thống như sau.

7.1 Ưu điểm

Hệ thống được triển khai theo quy trình tương đối rõ ràng và thuận tiện. Toàn bộ quá trình từ kết nối Google Drive, cài đặt thư viện, nạp mô hình, xây dựng API cho đến public server bằng ngrok đều có thể thực hiện nhanh chóng trong môi trường Google Colab. Điều này giúp rút ngắn đáng kể thời gian triển khai và phù hợp cho mục đích học tập, thử nghiệm và trình diễn.

Bên cạnh đó, API được xây dựng theo chuẩn HTTP REST nên có thể dễ dàng kiểm thử bằng nhiều công cụ khác nhau như Postman, trình duyệt, `curl` hoặc các đoạn mã Python/JavaScript. Hệ thống cũng hỗ trợ nhiều cách gửi dữ liệu đầu vào, bao gồm upload file ảnh trực tiếp và gửi ảnh dưới dạng base64, từ đó tăng tính linh hoạt trong quá trình sử dụng.

Một ưu điểm khác là cơ chế CORS đã được kích hoạt, cho phép API dễ dàng kết nối với các ứng dụng frontend ở domain khác. Điều này tạo điều kiện thuận lợi cho việc mở rộng hệ thống thành các ứng dụng web hoặc mobile trong tương lai. Ngoài ra, mã nguồn được tổ chức tương đối rõ ràng, tách biệt giữa phần tiền xử lý ảnh, định nghĩa API endpoint và logic dự đoán, giúp việc đọc hiểu và bảo trì trở nên dễ dàng hơn.

Trong phiên bản mở rộng của bài lab, hệ thống còn được bổ sung giao diện web đơn giản bằng HTML, CSS và JavaScript, giúp người dùng tải ảnh và xem kết quả dự đoán một cách trực quan hơn. Điều này làm tăng tính hoàn chỉnh và khả năng trình diễn của hệ thống.

7.2 Hạn chế

Mặc dù hệ thống đã hoạt động đúng chức năng, vẫn còn một số hạn chế nhất định. Trước hết, toàn bộ server đang phụ thuộc vào môi trường Google Colab, do đó chỉ có thể hoạt động khi notebook đang được mở và không bị ngắt phiên làm việc. Khi Colab bị timeout hoặc dừng chạy, API cũng sẽ ngừng hoạt động.

Bên cạnh đó, ngrok trong phiên bản miễn phí chỉ cung cấp URL tạm thời và địa chỉ này có thể thay đổi sau mỗi lần khởi động lại notebook. Điều này gây bất tiện nếu hệ thống cần tích hợp lâu dài với một frontend hoặc ứng dụng khác.

Về mặt mô hình, hệ thống hiện tại chỉ xử lý bài toán phân loại nhị phân giữa hai lớp `Cat` và `Dog`. Để mở rộng sang nhiều loại đối tượng hơn, cần xây dựng và huấn luyện lại mô hình phù hợp. Ngoài ra, API hiện chưa được tích hợp cơ chế xác thực như API key hoặc token, nên chưa đảm bảo tốt về mặt bảo mật nếu đưa vào sử dụng công khai.

Cuối cùng, do triển khai trên Colab miễn phí nên tài nguyên tính toán như CPU, GPU và RAM vẫn còn giới hạn. Điều này có thể ảnh hưởng đến tốc độ phản hồi nếu hệ thống phải xử lý nhiều yêu cầu đồng thời.

7.3 Bài học rút ra

Thông qua bài lab này, có thể thấy rõ quy trình triển khai một mô hình học máy hoặc học sâu thành một dịch vụ dự đoán thực tế. Quá trình này không chỉ dừng lại ở việc huấn luyện mô hình mà còn bao gồm các bước như chuẩn bị môi trường, nạp mô hình, xây dựng API, tiền xử lý dữ liệu đầu vào và public server để phục vụ truy cập từ bên ngoài.

Bài lab cũng cho thấy tầm quan trọng của bước tiền xử lý ảnh. Ảnh đầu vào cần được resize, chuẩn hóa và chuyển đổi đúng định dạng mà mô hình yêu cầu; nếu không, kết quả dự đoán có thể bị sai lệch hoặc không hoạt động đúng. Ngoài ra, việc xây dựng API bằng Flask giúp hiểu rõ hơn cách tạo ứng dụng web đơn giản, định nghĩa route, nhận request và trả về phản hồi ở dạng JSON.

Việc sử dụng ngrok để public server cục bộ ra internet cũng là một kinh nghiệm hữu ích trong các tình huống cần demo hoặc kiểm thử nhanh hệ thống AI. Đồng thời, thông qua phần phân tích mô hình, bài lab còn giúp củng cố hiểu biết cơ bản về kiến trúc CNN và vai trò của các lớp như Conv2D, BatchNormalization, MaxPooling, Dropout và Dense trong bài toán phân loại ảnh.

VIII. Kết luận

Bài lab đã triển khai thành công một mô hình phân loại ảnh Cat/Dog dưới dạng REST API bằng Flask trên môi trường Google Colab. Toàn bộ quy trình thực hiện bao gồm kết nối Google Drive để truy cập file mô hình đã huấn luyện, cài đặt các thư viện cần thiết, nạp mô hình CNN, phân tích thông tin đầu vào và đầu ra, xây dựng hàm tiền xử lý ảnh, phát triển các endpoint API và public hệ thống ra internet bằng ngrok.

Kết quả đạt được cho thấy hệ thống có khả năng tiếp nhận ảnh đầu vào từ phía người dùng dưới dạng file upload hoặc dữ liệu base64, thực hiện tiền xử lý phù hợp với yêu cầu của mô hình, sau đó đưa vào mô hình để dự đoán và trả về kết quả ở dạng JSON với nhãn phân loại và độ tin cậy tương ứng. Điều này cho thấy mô hình không chỉ dừng lại ở mức huấn luyện mà đã được đưa vào một dạng ứng dụng có thể sử dụng trong thực tế.

Ngoài phần backend API, bài lab còn được mở rộng bằng một giao diện web đơn giản viết bằng HTML, CSS và JavaScript. Giao diện này cho phép nhập địa chỉ API, tải ảnh lên và hiển thị kết quả dự đoán trực tiếp trên trình duyệt, góp phần làm cho hệ thống trực quan và dễ sử dụng hơn.

Bài lab là một bước nền tảng quan trọng trong quá trình phát triển các ứng dụng trí tuệ nhân tạo thực tế. Từ kết quả hiện tại, hệ thống có thể tiếp tục được mở rộng theo nhiều hướng như xây dựng frontend hoàn chỉnh hơn, phát triển ứng dụng mobile, triển khai trên máy chủ thực tế để tăng độ ổn định, hoặc tích hợp thêm các mô hình AI khác để giải quyết những bài toán phân loại phức tạp hơn.